

WEEK-4 ASSIGNMENT

Task Name

High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

Objective

To implement a frequent pattern mining engine from scratch using the FP-Growth algorithm and compressed FP-Trees. The system identifies frequent itemsets from transactional datasets and generates high-confidence association rules efficiently.

Description

The project focuses on building a high-performance market basket analysis system without relying on pre-built frequent-pattern mining libraries. The system dynamically constructs an FP-Tree from transaction data and uses node-link pointers to efficiently connect identical items.

The implementation recursively creates conditional pattern bases and conditional FP-Trees to discover frequent itemsets. Finally, the discovered patterns are used to generate association rules based on support, confidence, and lift.

Technologies Used

- Python
- FP-Growth Algorithm
- FP-Tree Data Structure
- Recursion
- Hash Tables / Dictionaries
- Node-Link Pointers
- Association Rule Mining

Key Features

1. Read and process transactional datasets.
2. Calculate item frequency and minimum support.
3. Remove infrequent items from transactions.
4. Sort items according to their frequency.
5. Dynamically construct a compressed FP-Tree.
6. Maintain node-link pointers for identical items.
7. Generate conditional pattern bases.
8. Recursively construct conditional FP-Trees.
9. Extract frequent itemsets.
10. Generate high-confidence association rules.
11. Calculate support, confidence, and lift.

12. Display useful market basket analytics.

Algorithm

Step 1: Load Transaction Data

The system accepts a dataset containing multiple transactions, where each transaction consists of a collection of purchased items.

Step 2: Calculate Item Frequencies

The frequency of every item is calculated by scanning all transactions.

Step 3: Apply Minimum Support

Items that do not satisfy the minimum support threshold are removed.

Step 4: Sort Transactions

The remaining items in every transaction are arranged in descending order of frequency.

Step 5: Construct FP-Tree

A compressed FP-Tree is created by inserting each processed transaction into the tree. Common prefixes are shared to reduce memory usage.

Step 6: Maintain Node Links

Nodes containing the same item are connected using node-link pointers. This allows efficient traversal during frequent pattern mining.

Step 7: Generate Conditional Pattern Bases

For each item, the system finds all prefix paths that lead to that item and constructs its conditional pattern base.

Step 8: Construct Conditional FP-Trees

Conditional pattern bases are used to recursively build conditional FP-Trees.

Step 9: Extract Frequent Itemsets

Frequent itemsets are generated recursively by combining suffix items with conditional patterns.

Step 10: Generate Association Rules

Frequent itemsets are converted into association rules. Rules are evaluated using confidence and lift.

Sample Python Implementation

```
from collections import defaultdict

class FPNode:
    def __init__(self, item, count, parent):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.link = None

def update_header(item, node, header_table):
    if header_table[item][1] is None:
        header_table[item][1] = node
    else:
        current = header_table[item][1]
        while current.link is not None:
            current = current.link
        current.link = node

def insert_transaction(transaction, node, header_table, count=1):
    if not transaction:
        return

    item = transaction[0]

    if item in node.children:
        child = node.children[item]
        child.count += count
    else:
        child = FPNode(item, count, node)
        node.children[item] = child
        update_header(item, child, header_table)

    insert_transaction(transaction[1:], child, header_table, count)

def build_fp_tree(transactions, min_support):
    frequency = defaultdict(int)

    for transaction in transactions:
        for item in transaction:
            frequency[item] += 1

    frequency = {
        item: count
        for item, count in frequency.items()
        if count >= min_support
    }

    if not frequency:
        return None, None
```

```

header_table = {
    item: [count, None]
    for item, count in frequency.items()
}

root = FPNode(None, 0, None)

for transaction in transactions:
    filtered = [
        item for item in transaction
        if item in frequency
    ]

    filtered.sort(
        key=lambda item: frequency[item],
        reverse=True
    )

    if filtered:
        insert_transaction(
            filtered,
            root,
            header_table
        )

return root, header_table

def display_tree(node, level=0):
    for child in node.children.values():
        print(
            " " * level +
            f"{child.item} ({child.count})"
        )
        display_tree(child, level + 1)

# Sample Transaction Dataset
transactions = [
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter"],
    ["Milk", "Bread"],
    ["Milk", "Butter"],
    ["Bread", "Butter", "Eggs"],
    ["Milk", "Bread", "Eggs"],
    ["Milk", "Bread", "Butter"]
]

minimum_support = 2

root, header_table = build_fp_tree(
    transactions,
    minimum_support
)

print("Compressed FP-Tree:")
display_tree(root)

print("\nHeader Table:")
for item, data in header_table.items():
    print(item, ":", data[0])

```

Association Rule Metrics

Support

Support measures how frequently an itemset occurs in the dataset.

Formula:

$\text{Support}(X) = \frac{\text{Number of transactions containing } X}{\text{Total number of transactions}}$

Confidence

Confidence measures how often item Y appears when item X appears.

Formula:

$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$

Lift

Lift measures the strength of an association between two items.

Formula:

$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$

A lift value greater than 1 generally indicates a positive association between the items.

Sample Output

Compressed FP-Tree:

```
Bread (6)
  Milk (4)
    Butter (2)
    Eggs (1)
  Eggs (1)
  Butter (2)
    Eggs (1)
Butter (1)
```

Header Table:

```
Milk : 4
Bread : 6
Butter : 5
Eggs : 2
```

Advantages

- Uses a compressed tree structure.

- Reduces the need to repeatedly scan the complete transaction database.
- Uses node links for efficient item traversal.
- Supports recursive conditional FP-Tree construction.
- Can handle large transactional datasets efficiently.
- Useful for discovering customer purchasing patterns.
- Helps generate meaningful product association rules.

Applications

- Supermarket market basket analysis
- E-commerce product recommendations
- Customer purchase analysis
- Cross-selling and up-selling
- Retail inventory planning
- Personalized recommendations
- Sales pattern discovery

Conclusion

The High-Performance Market Basket Analytics system demonstrates how FP-Growth can efficiently discover frequent patterns from transactional data. The compressed FP-Tree minimizes redundant storage by sharing common transaction prefixes, while node-link pointers provide efficient access to related nodes. Conditional pattern bases and recursive conditional FP-Trees allow frequent itemsets to be mined without generating a large number of candidate sets. The resulting frequent patterns can further be converted into high-confidence association rules to support practical business decision-making.